

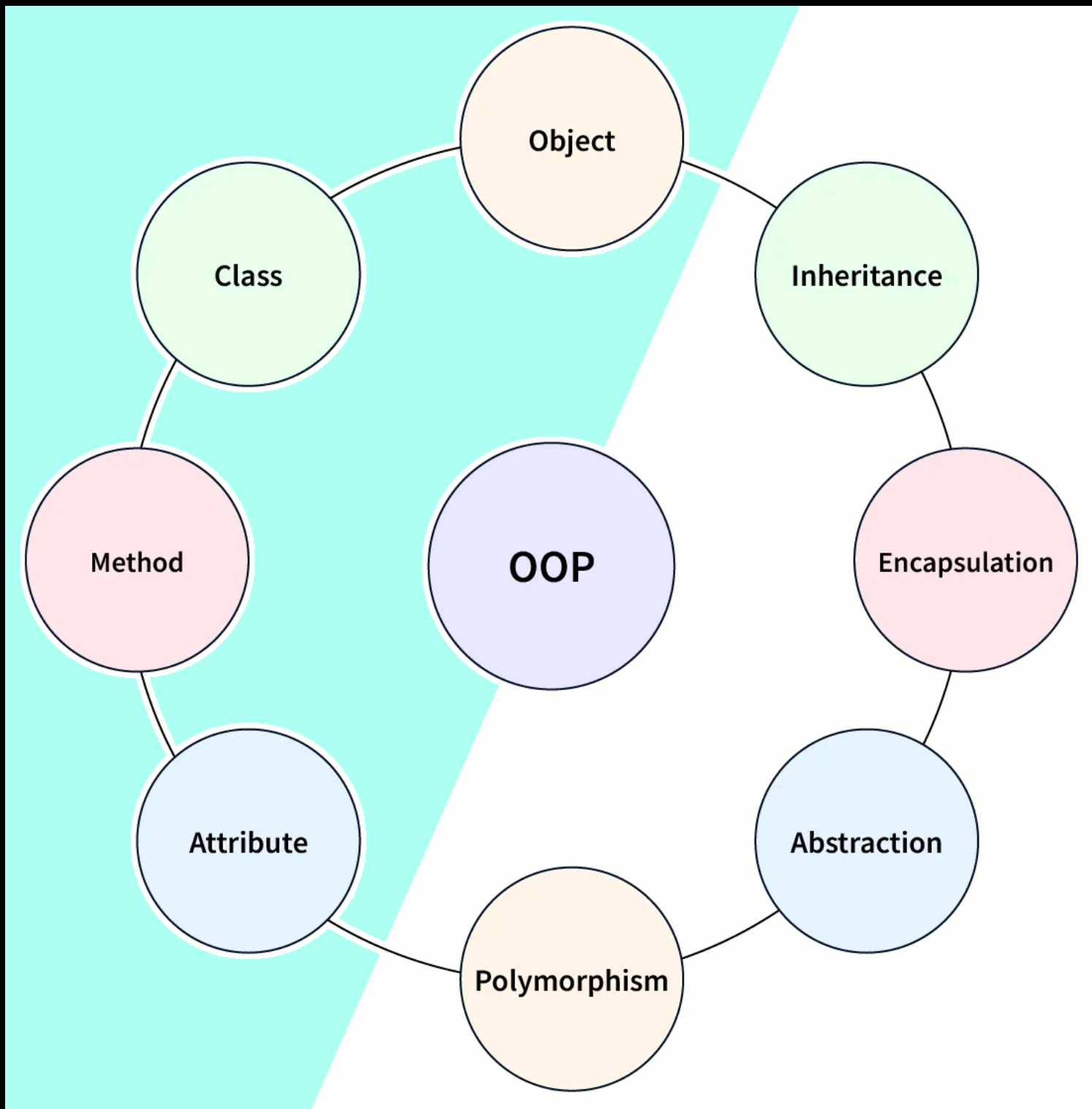
ANÁLISIS DE SISTEMAS

Modelado de Objetos

OBJECT ORIENTED



PROGRAMMING

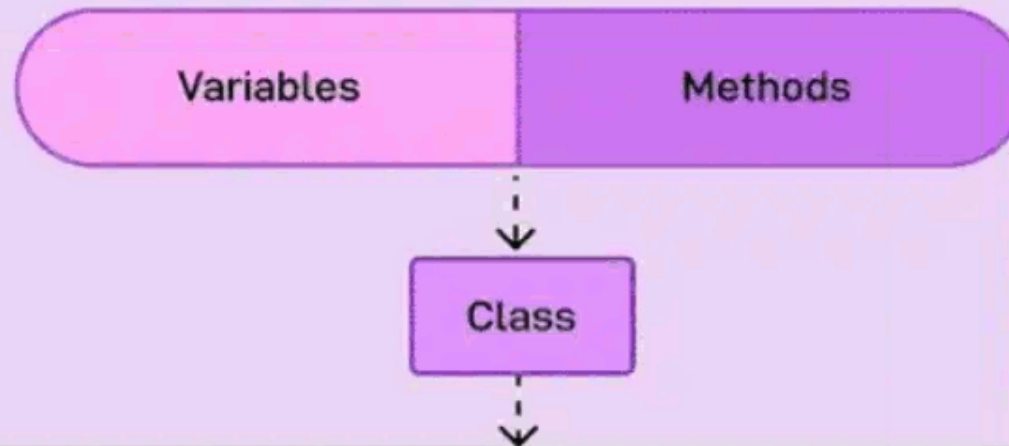


Encapsulamiento



Encapsulation

- Process of wrapping code and data together into a single unit. It means each object in the code should control its own state



```
public class Car {  
    // private variable  
    private String name;  
  
    // getter method for name  
    public String getName() {  
        return name;  
    }  
  
    // setter method for name  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Variables

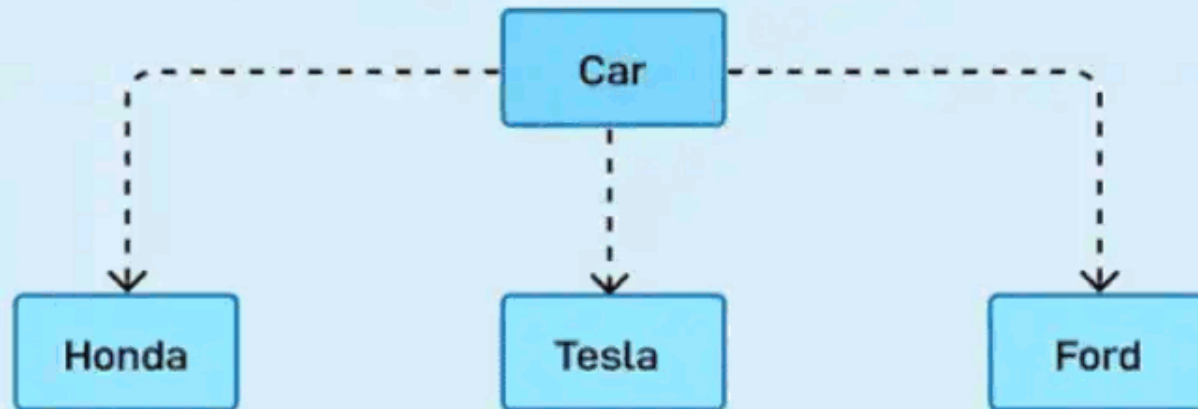
Methods

Abstracción



Abstraction

- Process of hiding implementation details and exposing only the functionality to the user.



```
public abstract class Car {  
    public abstract void stop();  
}
```

Abstract Class and Method

```
// Concrete class
```

```
public class Honda extends Car {  
    @Override public void stop()  
    {  
        System.out.println("Honda::Stop");  
        System.out.println(  
            "Mechanism to stop the car using break");  
    }  
}
```

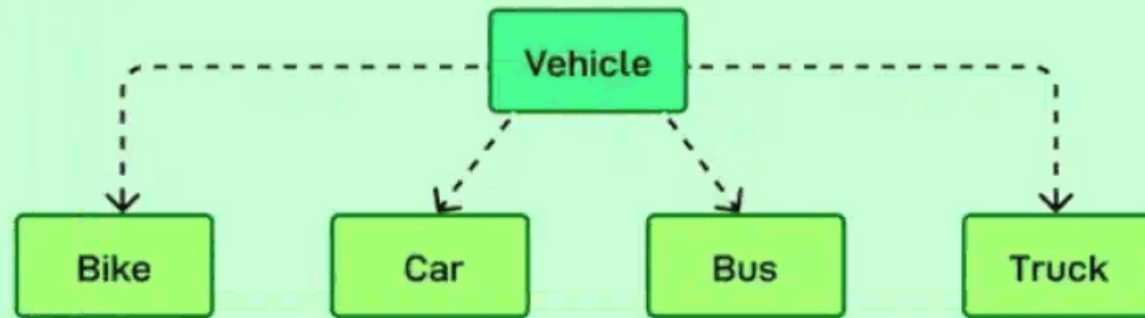
Providing Implementation

Herencia



Inheritance

- Process of one class inheriting properties and methods from another class. Used to represent the IS-A relationship between objects



Base Class

```
class Vehicle {  
    protected String brand;  
  
    // Constructor  
    public Vehicle(String brand)  
    {  
        this.brand = brand;  
    }  
  
    // Method to be overridden  
    public String describe() {  
        return "This is a  
vehicle of brand: " + brand;  
    }  
}
```

Derived Class

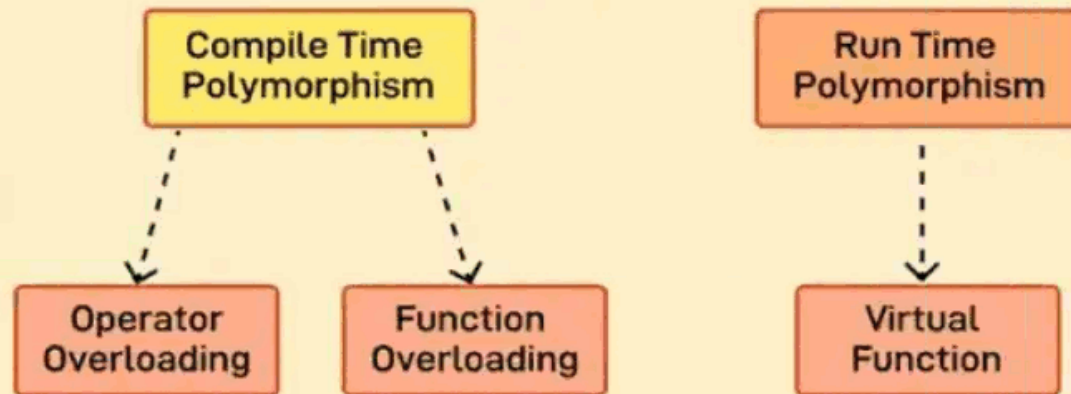
```
class Car extends Vehicle {  
    private int doors;  
  
    // Constructor  
    public Car(String brand, int  
doors) {  
        super(brand);  
        this.doors = doors;  
    }  
  
    // Override the describe method  
    @Override  
    public String describe() {  
        return "This is a car of  
brand: " + brand  
            + " with " + doors +  
" doors."  
    }  
}
```

Polimorfismo



Polymorphism

- Ability to perform many things in many ways. When two types share an inheritance chain, they can be used interchangeably with no errors. This also means using parents like their children



Definition

```
class Vehicle {
    public void start() {
        System.out.println("The
vehicle starts.");
    }
}

// Derived class 1
class Car extends Vehicle {
    @Override
    public void start() {
        System.out.println("The
car starts with a key.");
    }
}
```

Usage

```
Vehicle myVehicle; // Reference
of type

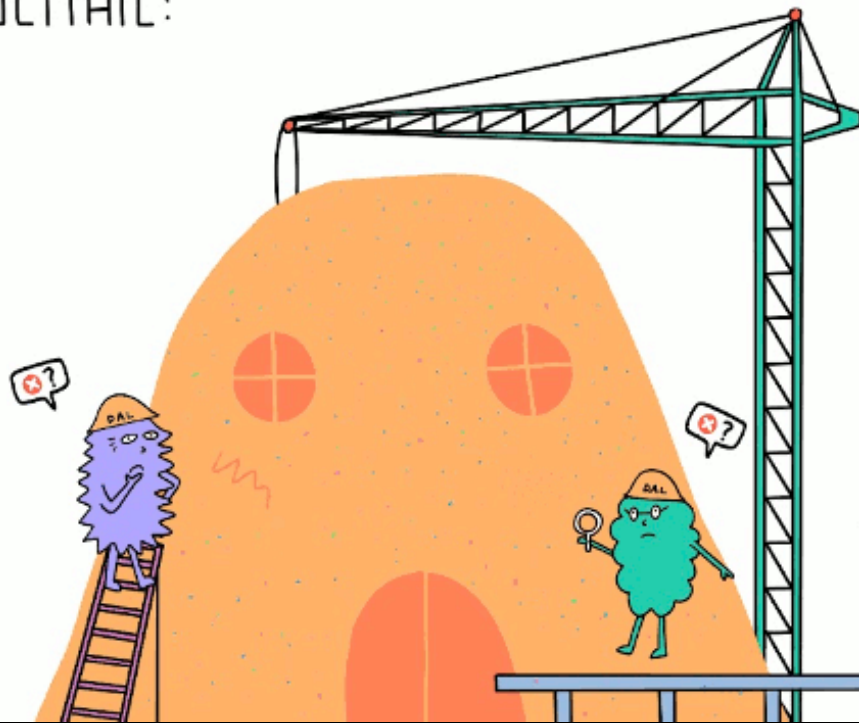
// Assign Car object to the
Vehicle reference
myVehicle = new Car();

myVehicle.start();
```

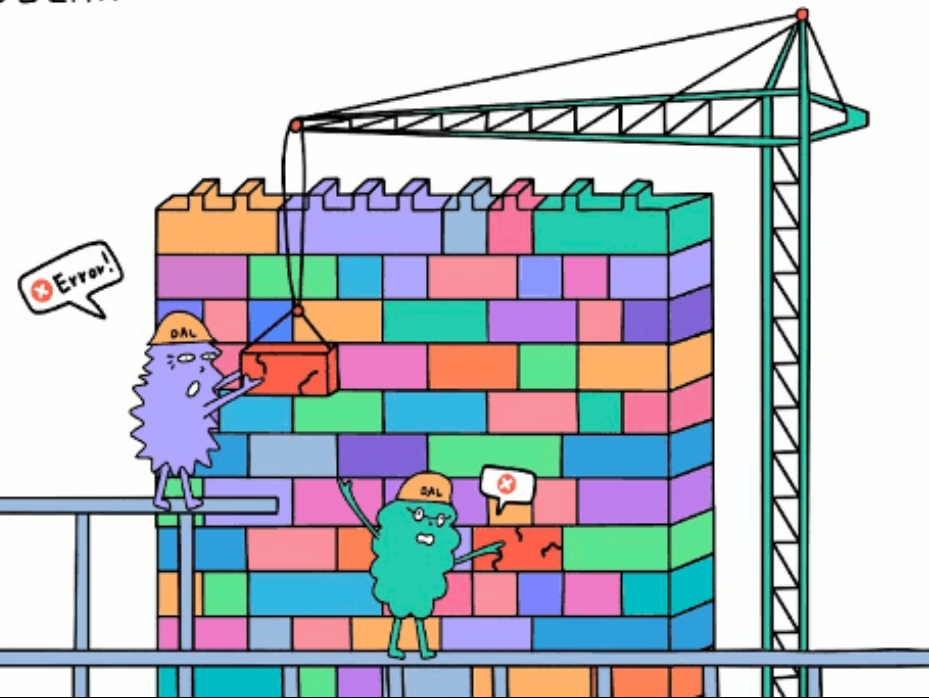
Análisis Orientado a Objetos

- **Modular**

MONOLITHIC:



MODULAR:

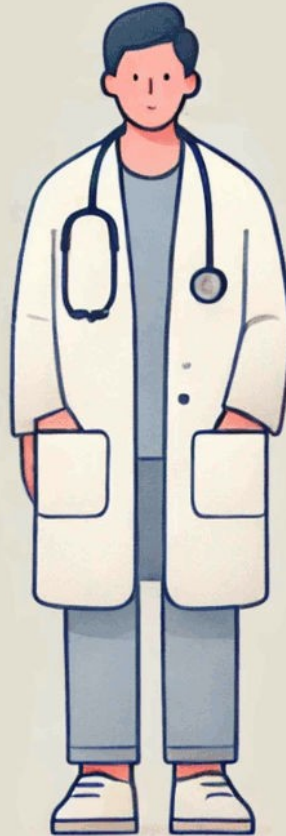


Análisis Orientado a Objetos

- Modular
- **Objetos**



Paciente



Doctor

A spiral-bound calendar with a red cover. It shows a grid with days of the week and dates. The word 'Patient' is written in the first row, 'Doctor' in the second, and 'Object' in the third. The date 19 is highlighted in red.

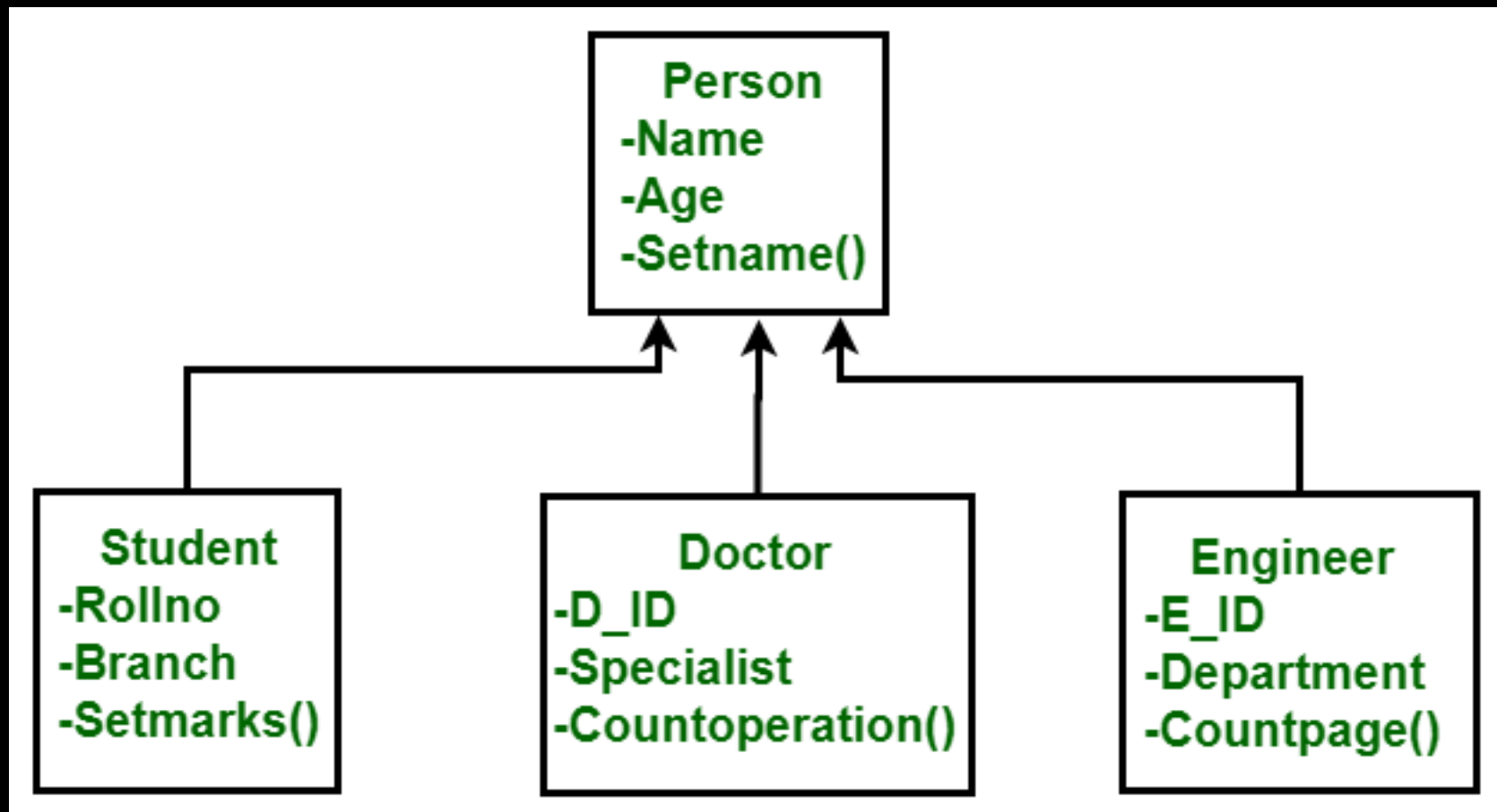
Day	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Patient	1	2	3	4	5	6	7
	8	9	10	11	12	13	14
Doctor	15	16	17	18	19	20	21
	22	23	24	25	26	27	28
Object	29	30	31				



Turno

Análisis Orientado a Objetos

- Modular
- Objetos
- **Modelo**



Objetos

(sustantivos)

Atributos

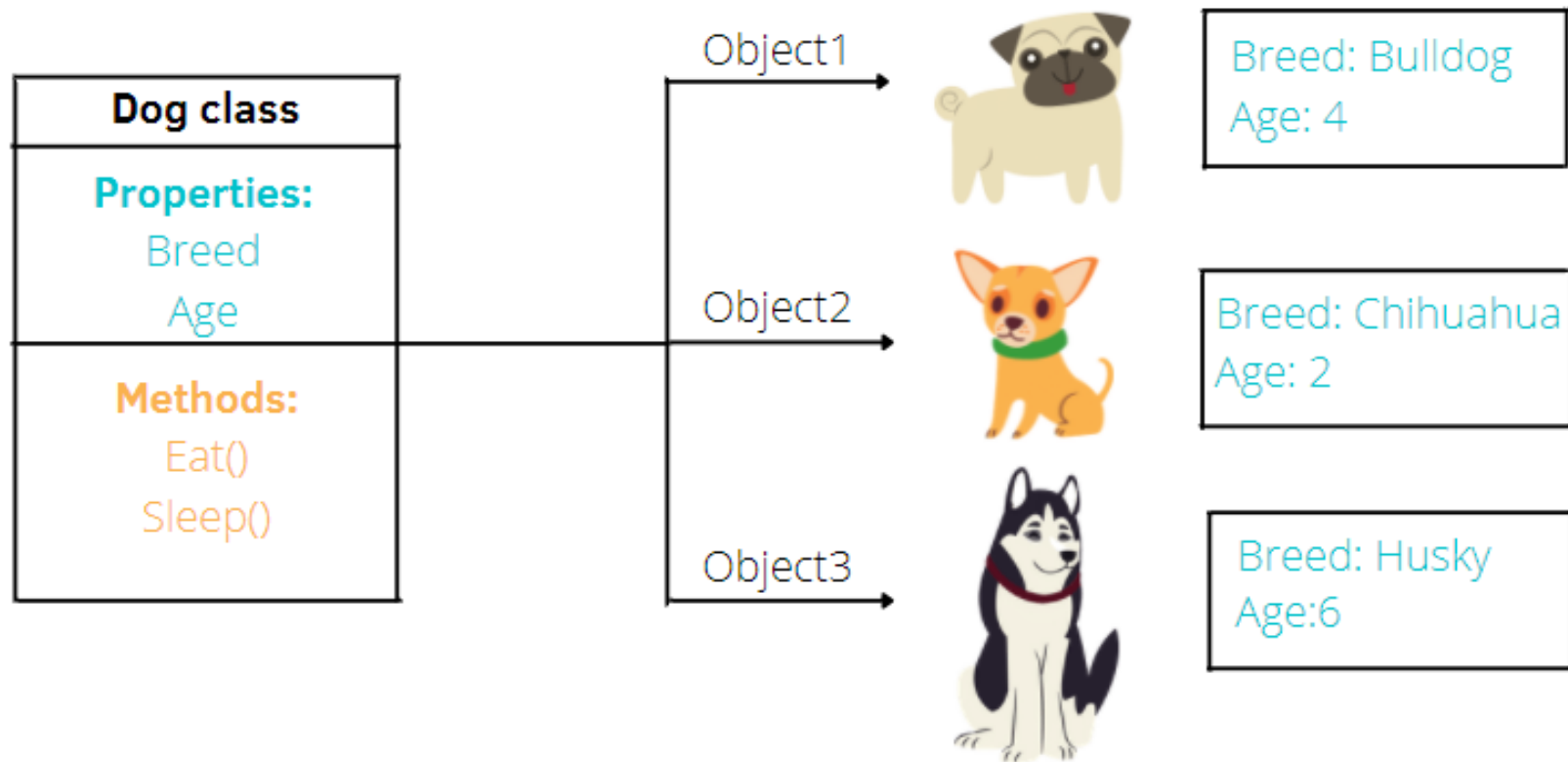
(adjetivos)

Métodos

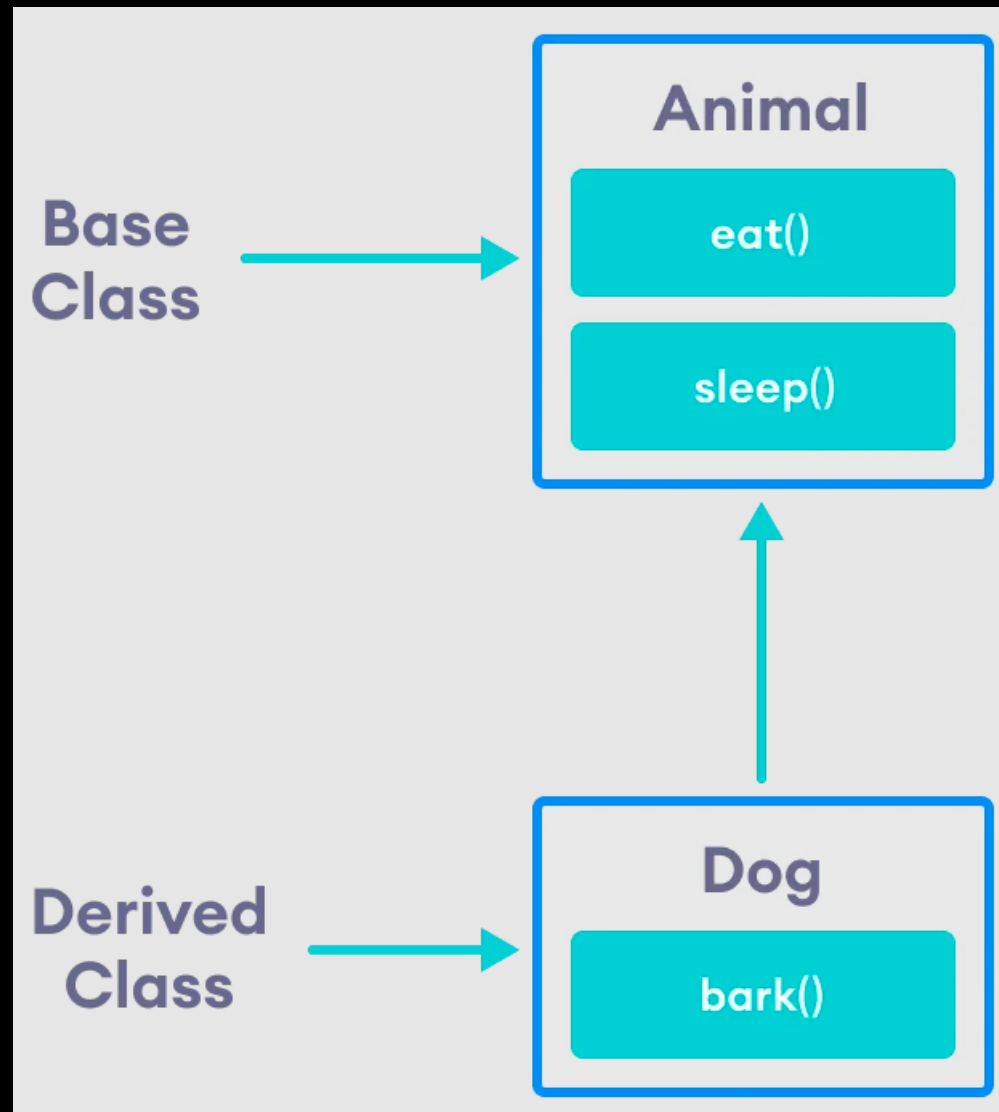
(verbos)

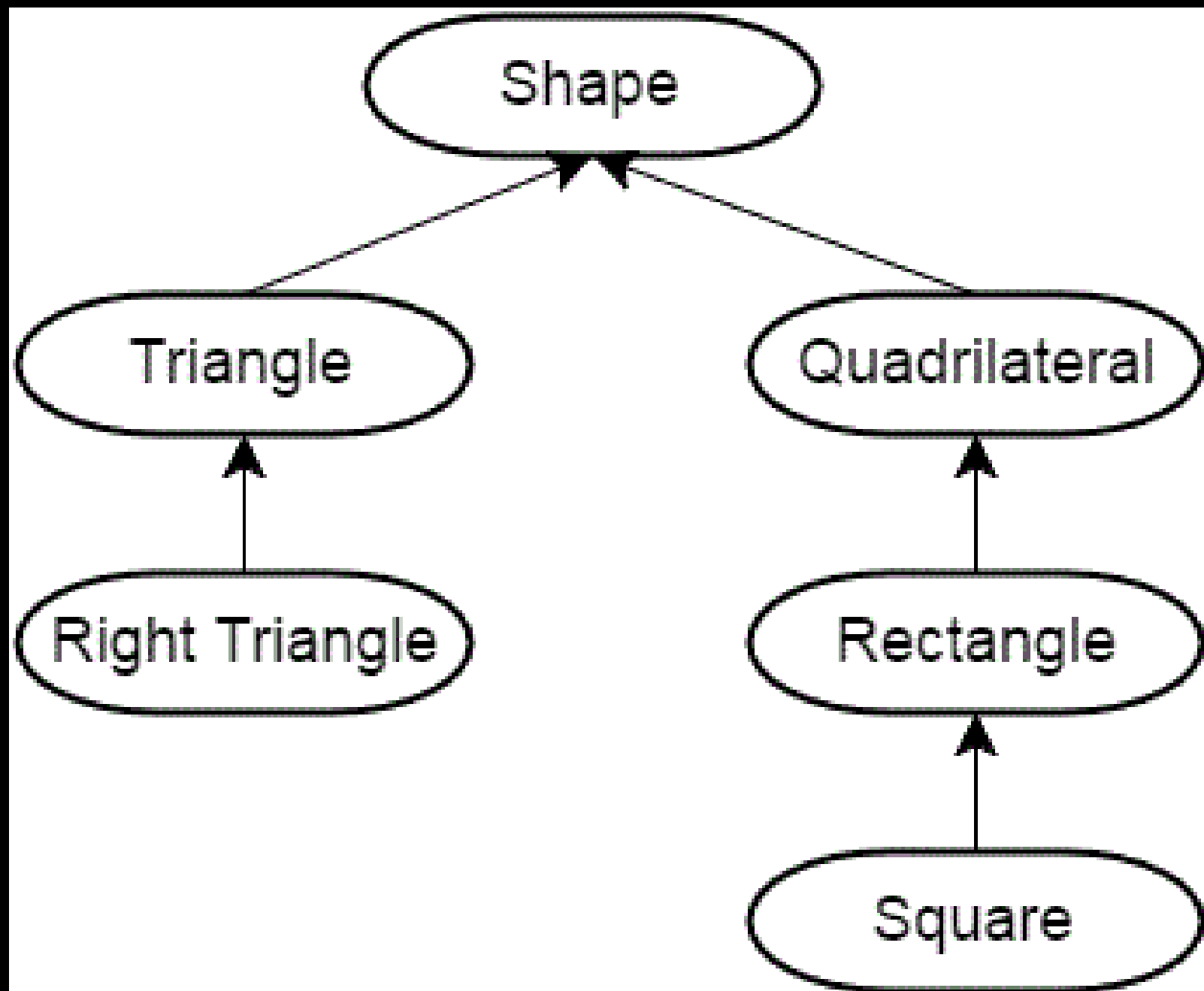
Mensajes

Classes



Relaciones entre Objetos y Clases

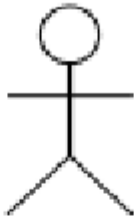
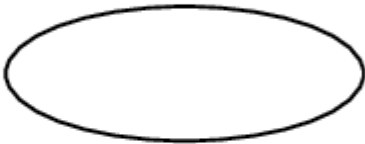
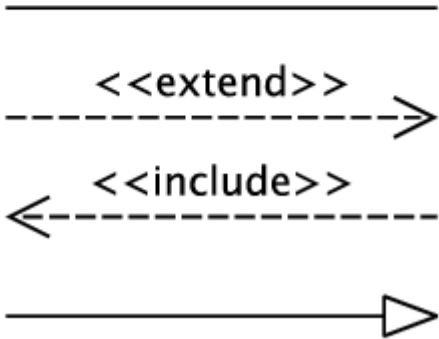




Lenguaje Unificado de Modelado

Lenguaje Unificado de Modelado

- **Modelado de Casos de Uso**

Symbol	Reference Name
	Actor
	Use case
	Relationship

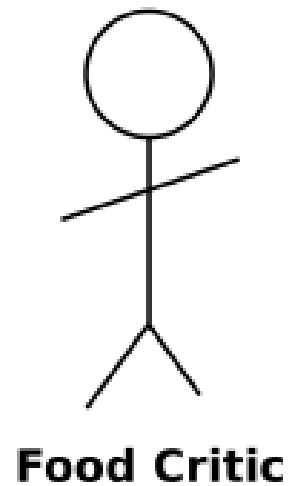
Use Case description – Retrieve New Password

High-level description					
	A NetCom Personal Customer has lost the password and requests a new one. Can be used to change the password. The function is used by other NetCom web-portal.				
Actors					
	- NetCom Personal Customer starts the use case, called user in the use case. - Profile server, the data storage of profile data. - NetCom web-portal, the originating system for the request, e.g. MinSide, Mother or mms.netcom.no. - SMSC (SMS center)				
Pre-conditions					
	<table border="1" style="width: 100%; border-collapse: collapse;"> <tr> <td style="width: 10%; padding: 5px;">P1.</td><td style="padding: 5px;">The user has accessed this system from another web-based self-service and information systems at NetCom.</td></tr> </table>	P1.	The user has accessed this system from another web-based self-service and information systems at NetCom.		
P1.	The user has accessed this system from another web-based self-service and information systems at NetCom.				
Post-conditions					
	Undefined				
Flow of events					
	<table border="1" style="width: 100%; border-collapse: collapse;"> <tr> <td style="width: 10%; padding: 5px;">1.</td><td style="padding: 5px;">The use-case starts when the user requests to retrieve a new password. If msisdn is not given in the URL, the user is asked for msisdn.</td></tr> <tr> <td style="padding: 5px;">2.</td><td style="padding: 5px;">The system gets the secret question and answer from the profile</td></tr> </table>	1.	The use-case starts when the user requests to retrieve a new password. If msisdn is not given in the URL, the user is asked for msisdn.	2.	The system gets the secret question and answer from the profile
1.	The use-case starts when the user requests to retrieve a new password. If msisdn is not given in the URL, the user is asked for msisdn.				
2.	The system gets the secret question and answer from the profile				

Lenguaje Unificado de Modelado

- Modelado de Casos de Uso
- **Diagramas de Casos de Uso**

Restaurant (simplified)

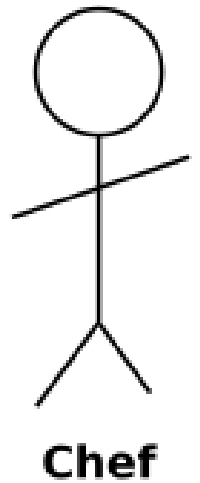


Eat Food

Pay for Food

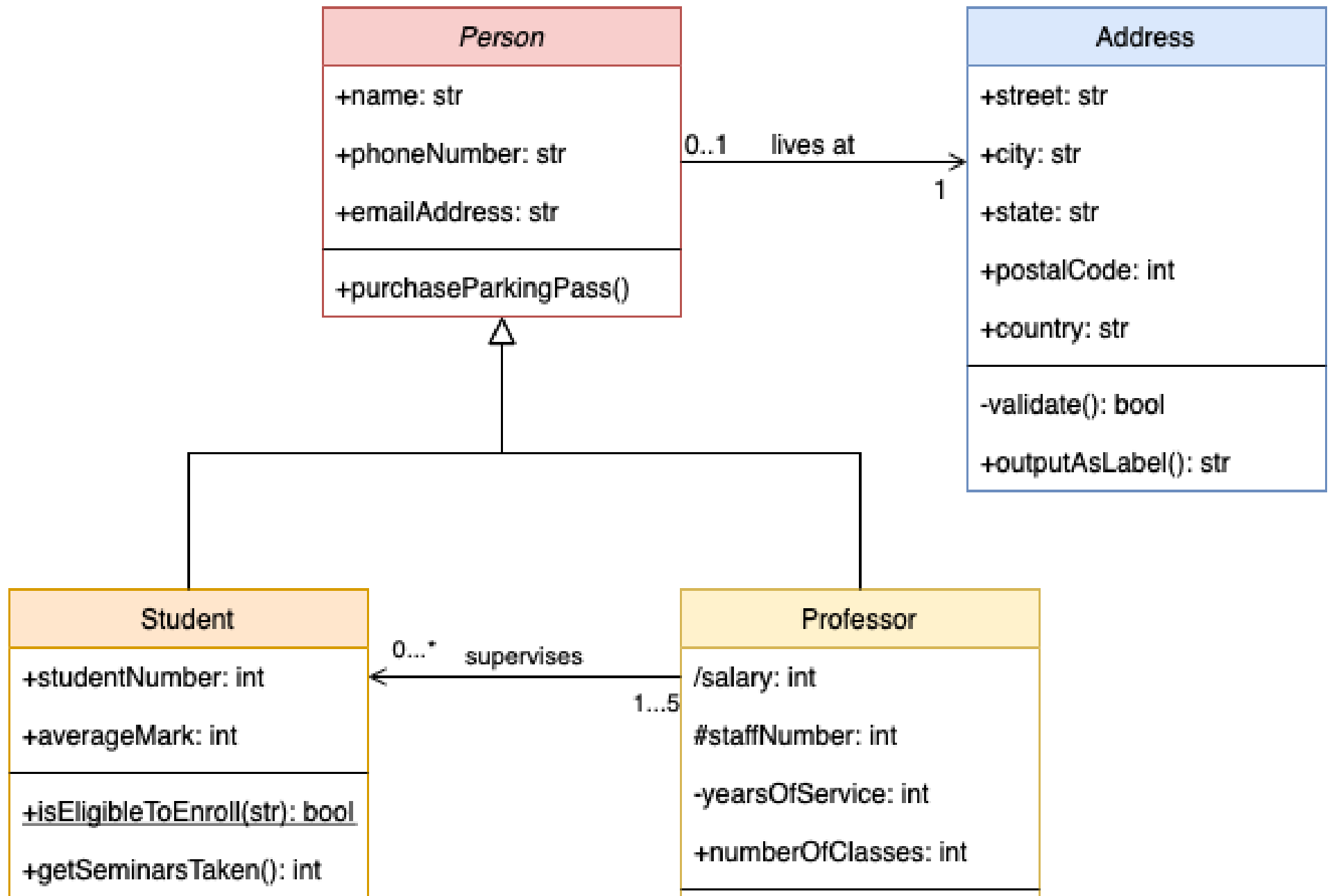
Drink Wine

Cook Food



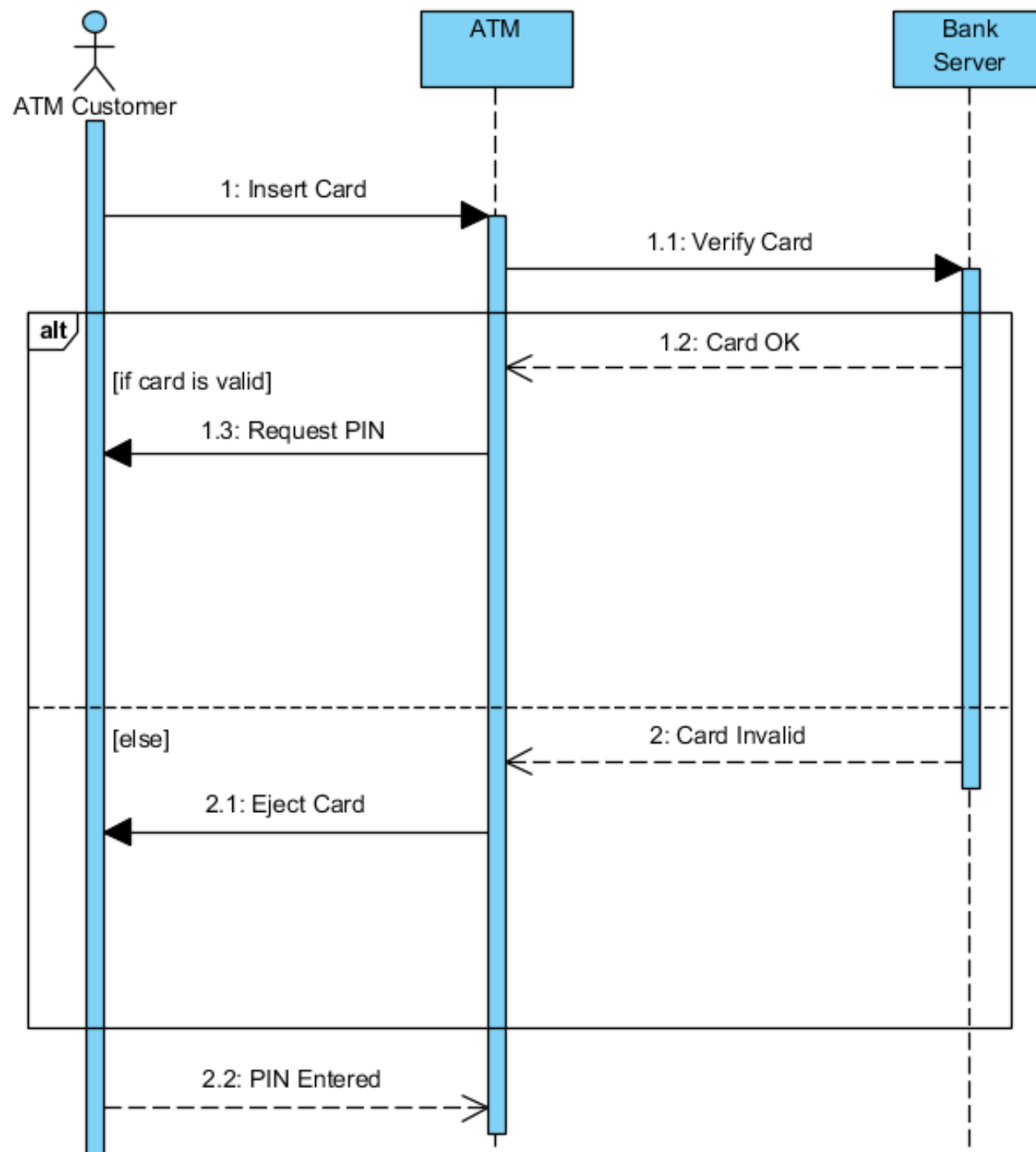
Lenguaje Unificado de Modelado

- Modelado de Casos de Uso
- Diagramas de Casos de Uso
- **Diagramas de Clases**



Lenguaje Unificado de Modelado

- Modelado de Casos de Uso
- Diagramas de Casos de Uso
- Diagramas de Clases
- **Diagramas de Secuencias**

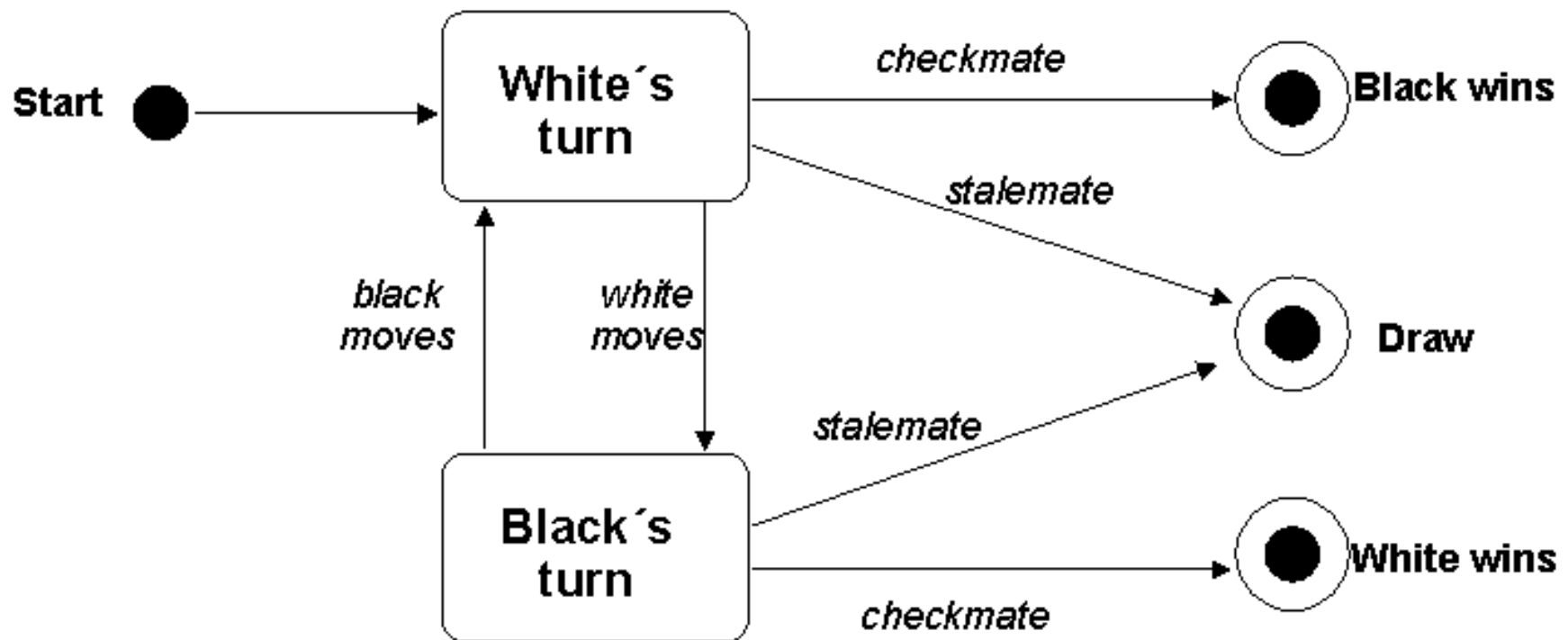


Lenguaje Unificado de Modelado

- Modelado de Casos de Uso
- Diagramas de Casos de Uso
- Diagramas de Clases
- Diagramas de Secuencias
- **Diagramas de Transición de Estado**

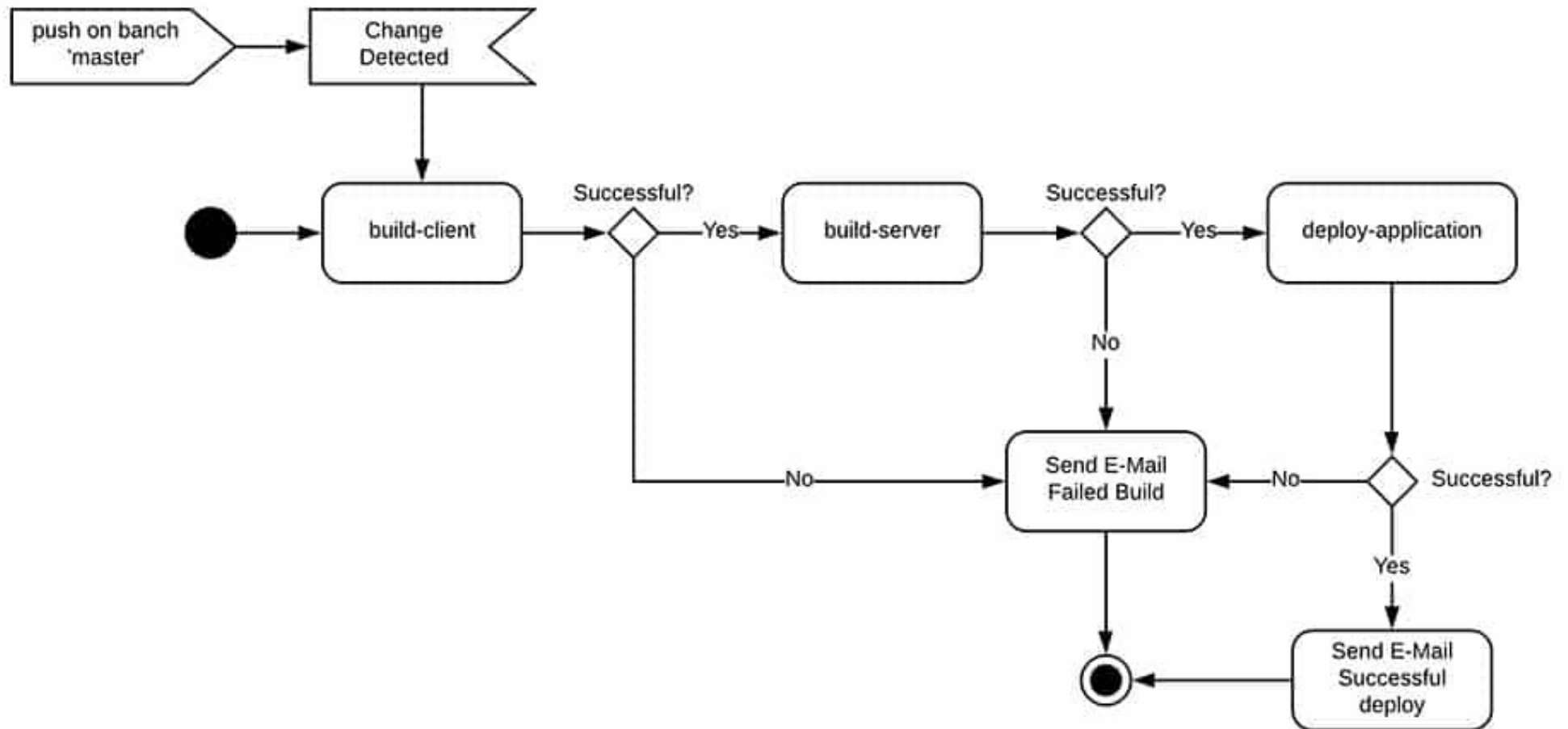
UML State Diagram - example

Chess game



Lenguaje Unificado de Modelado

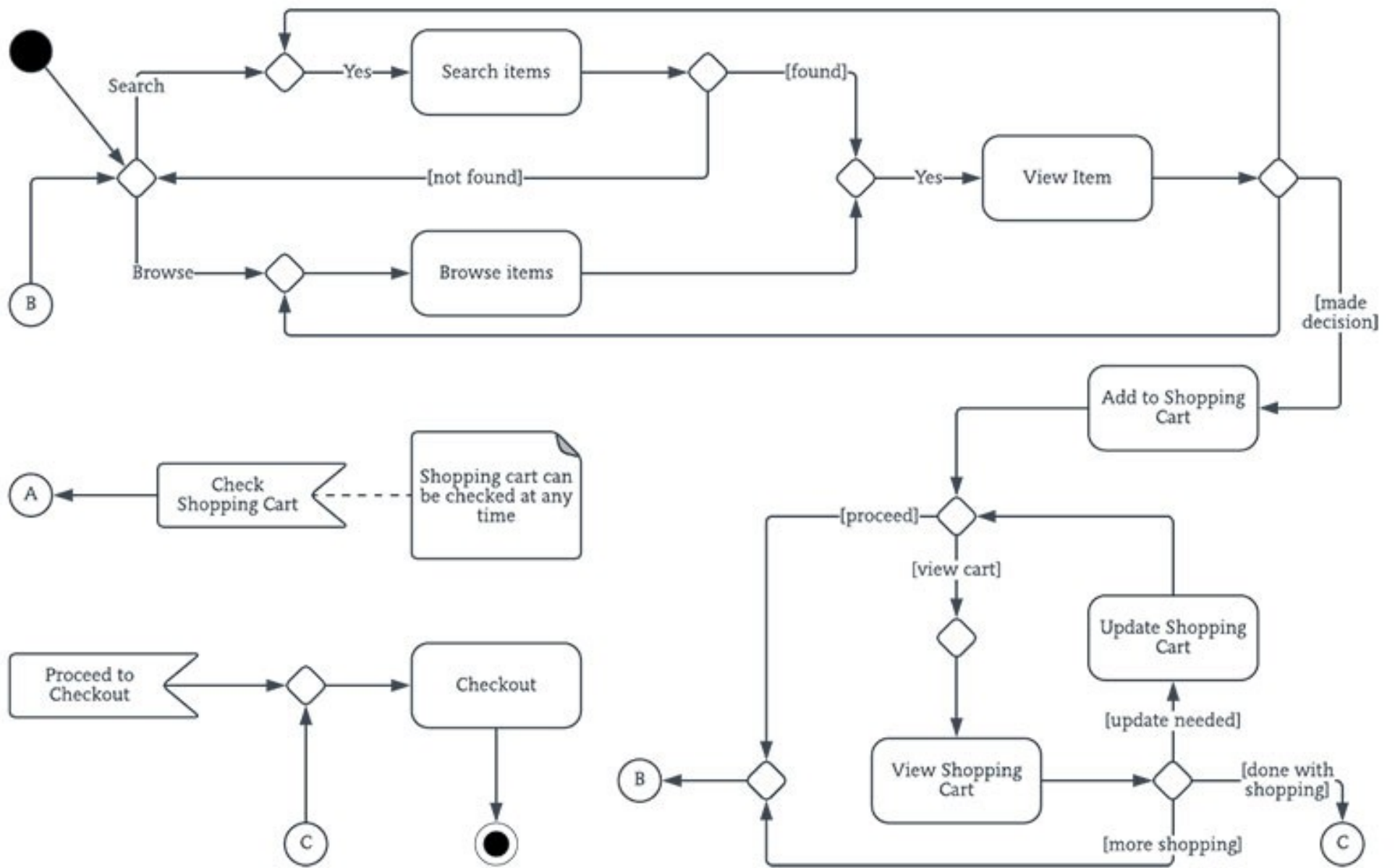
- Modelado de Casos de Uso
- Diagramas de Casos de Uso
- Diagramas de Clases
- Diagramas de Secuencias
- Diagramas de Transición de Estado
- **Diagramas de Actividades**



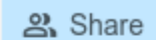
ACTIVITY DIAGRAM

Lenguaje Unificado de Modelado

- Modelado de Casos de Uso
- Diagramas de Casos de Uso
- Diagramas de Clases
- Diagramas de Secuencias
- Diagramas de Transición de Estado
- Diagramas de Actividades
- **Modelado del Proceso de Negocio**



Herramientas

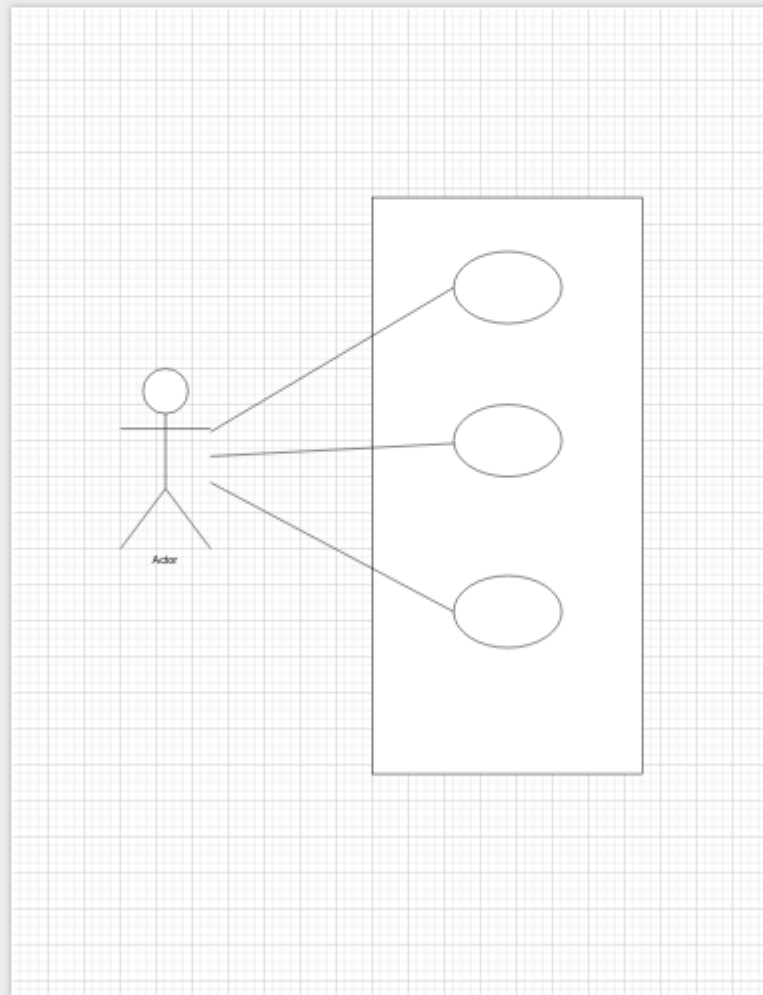


Unsaved changes. [Click here to save.](#)



Drag elements here

+ More Shapes



Style

☐ Shadow

 Guides

☒ Portrait ☐ Landscape

Clear Default Style